DNSC 4280 Section 10 (Fall 2025): Machine Learning

Project Report: Jonathan Beck

Table Of Contents

Abstract…….....3

1. Problem Statement and Motivation.....3

2. Data Selection and Understanding ….....4

3. Data Preparation and Feature Engineering.....5

    3.1 TF-IDF Vectorization for Linear Models.....5

    3.2 Tokenization for RNN Models.....5

    3.3 Tokenization for Transformer Models …….....6

4. Modeling Selection and Best Model Results….....6

    4.1 Multinomial Naïve Bayes.....7

    4.2 Regularized Multinomial Logistic Regression….....7

    4.3 Recurrent Neural Network (LSTM variant).....9

    4.4 Transformer (RoBERTa)..... 11

5. Discussion.....13

    5.1 Text Field Choice……..13

    5.2 Why Frequent Global N-Grams Were not Removed……13

    5.3 Training a Traditional Artificial MLP Neural Network…..13

6. Business-oriented Recommendations..... 13

    6.1 Recommended Baseline Model for Operational Deployment……14

    6.2 Considering Advanced Architectures……..14

7. Conclusion……14

Abstract

This project develops and evaluates a set of text classification models to predict the topical category of new articles using the AG News dataset from Kaggle. The predictive task is four-class multinomial classification problem, where each article belongs to one of: World, Sports, Business, or Sci/Tech. The analysis ranges from baseline linear models (Naïve Bayes and regularized Logistic Regression using TF-IDF features) to more advanced architectures, including a Recurrent Neural Network (RNN) and a pretrained (RoBERTa) Transformer fine-tuned for this task.

Across all experiments, models using both the article's title and description as predictors consistently achieved optimal classification performance compared with versions in which only a single field was used. Naïve Bayes and Logistic Regression both achieved strong baseline performance (test accuracy above 91%) when trained on TF-ID representations of the combined text. A simple LSTM-based RNN further validates that sequential modeling can outperform the baseline models marginally for this classification task. The best overall performance was obtained by designing a RoBERTA-based Transformer classifier, which reaches approximately 94% test accuracy with balanced classification metrics demonstrated across all four classes.

The key takeaway is that for general documentation classification tasks, modern transfer learning models provide a meaningful performance uplift over traditional linear baselines, but at a significantly higher computational cost. Thus, in some settings, a regularized logistic regression with TF-IDF features has the capabilities to offer an attractive blend of accuracy, interpretability, and training speed, while Transformer-based models are perhaps best reserved for more complex tasks (e.g., translation) or scenarios where the marginal gains in accuracy justify the additional training time and computational expense.

1. Problem Statement and Motivation

In this project, the modeling objective is to successfully assign each article to one of four classes: World, Sports, Business, or Science/Technology. In the modern business domain, accurate text classification reduces manual labeling efforts, enhances content recommendation, news aggregation, and trend monitoring endeavors, and improves user experience while surfing the web, browsing articles by subject matter, and conducting research.

The goal in this project is not to maximize state-of-the-art performance, but rather to establish 1) a clean and auditable preprocessing pipeline, 2) defensible baseline accuracy, and 3) evidence concerning which text fields are most informative (Title vs Description vs both). Specifically, the objective is to answer the practical question: *"Given a large stream of news*

articles, what modeling approach provides the best balance between predictive performance, computational cost, and implementation complexity for topic classification?

2. Data Selection and Understanding

The AG News dataset was obtained through Kaggle, containing two CSV's: a train.csv (120,000 rows and 3 columns), and a test.csv (7,600 rows and 3 columns). Each record includes three fields: 1) a class label represented by an integer; 2) a title that consists of a short string of characters; and 3) a description string that details a short summary of the article (Figure 1). The four class labels were represented as follows: 1=World, 2=Sports, 3=Business, and 4=Sci/Tech.

```
train_data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 120000 entries, 0 to 119999
Data columns (total 3 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   class        120000 non-null  int64
 1   title        120000 non-null  object
 2   description  120000 non-null  object
dtypes: int64(1), object(2)
memory usage: 2.7+ MB
```

```
test_data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7600 entries, 0 to 7599
Data columns (total 3 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   class        7600 non-null   int64
 1   title        7600 non-null   object
 2   description  7600 non-null   object
dtypes: int64(1), object(2)
memory usage: 178.3+ KB
```

Figure 1

Initial EDA indicated no null values in any capacity, no duplicate records, and all three features consisting of the expected datatypes (Title and Description=object; class label=integer) across both datasets. The class distribution was perfectly balanced (25% per class) in both train and test files, which negated concerns about class imbalance and made accuracy a meaningful first-line evaluation metric (Figure 2).

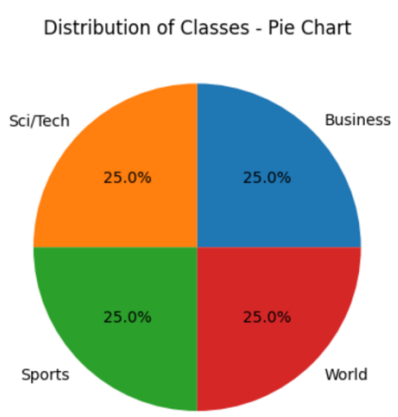

Figure 2

3. Data Preparation and Feature Engineering

The preprocessing phase began with standardizing the column names to induce a more efficient queries and analytics: Class Index → class; Title → title; and Description → description. Moreover, a class_name column was created for the purpose of more interpretable graphs and visualizations. Three predictive feature variations were constructed to test the predictive value of each text dimension: 1) Title only; 2) Description only; and 3) Title concatenated with Description (title+description). Overall, this structure ensured a controlled environment to compare the gain in predictive signal offered by each text dimension.

3.1 TF-IDF Vectorization for Linear Models

For the Naïve Bayes and Logistic Regression models, the text was transformed into a Term Frequency-Inverse Document Frequency (TF-IDF) representation that effectively converts raw text into weighted numerical features by capturing how important each word is to a document and down-weighting it based on how common the token is across the entire corpus.

The vectorization process involved several critical steps. First, the text was tokenized into unigrams and bigrams, enabling the model to infer correlations between individual tokens, two-word sequences, and article subject matter. Next, as previously stated, the TF-IDF algorithm down-weights scores for n-grams that appeared frequently across the bundle of articles, and up-weighting those specific to the article. Finally, each document vector was L2-normalized to ensure that classification decisions were driven by content similarity rather than text length, and to prevent scores from being skewed toward longer text descriptions. Moreover, English stop words, such as “and”, “or”, and “the” were removed, and the vocabulary size was capped between 20,000 and 40,000 tokens, depending on model selection, to control dimensionality and improve generalization and runtime.

3.2 Tokenization for RNN Models

For the Recurrent Neural Network (RNN), the preprocessing pipeline shifted from TF-IDF to sequence-based tokenization, given the RNN’s need to process text as an ordered sequence rather than a bag of words.

Using Keras’ TextVectorization class, text was tokenized (spit at whitespace), and punctuation was removed and all characters lowercased as a normalization procedure. Token frequencies were then globally counted and ranked by frequency, and each token was then assigned an ID, in descending frequency order. I then built a vocabulary of the 30,000 most frequent tokens in the training data as a regularization control, to ensure the vocabulary only consisted of the 30K most frequent words across all news articles. Moreover, this included stop words, as RNNs (and Transformers) benefit from full context, word order, and grammatical

structure of token inputs prior to constructing embeddings. Every article was then converted into a sequence of up to 200 tokens as a generalization decision to predetermine how many tokens the RNN was allowed to observe for each article at each time step before constructing the article's embedding. Shorter text descriptions were padded with zeros, and longer texts were truncated. This design choice was implemented to find a balance between computational efficiency and optimal linguistic classification, as news articles typically reveal their topical signal early, and additional length provides diminishing marginal returns in predictive and classification-oriented tasks.

The resulting tokens for each article were then fed into an Embedding layer, which learned the dense vector representations of each article's text token during training. The embedding layer is critical, as it allows the RNN to capture semantic relationships between words in each sequence, something linear-based models incorporating TF-IDF matrices simply cannot do. The LSTM (Long-Short Term Memory) layer then processed these embeddings sequentially before outputting a learned single vector representation of the whole sequence for multinomial classification.

Overall, this neural architecture provides a richer representation of the articles text than linear models, at the cost of training speed and increased design complexity.

3.3 Tokenization for Transformer Models

Transformer-based models, such as RoBERTA, required an alternative preprocessing pipeline, due to their attention-based procedural structure. For example, instead of the simple whitespace tokenization technique used in the prior models, subword segmentation was applied to handle morphological variations and out-of-vocabulary terms more effectively.

After words were tokenized and provided subword token IDs, based on the vocab learned from the pretrained and fixed RoBERTA model, sequences were passed through a StartEndPacker layer to enforce a fixed maximum sequence length of 128 tokens. These steps, similar to the process in RNN preprocessing, pad shorter articles with fake tokens to enforce all inputs conform to a standard shape, truncate longer ones, and add special start and end tokens to each sequence. Moreover, a padding mask is provided that instructs the Transformer to ignore padded token positions during the attention phase to ensure the self-attention mechanism focuses only on meaningful article content.

4. Modeling Selection and Best Model Results

A layered modeling strategy was deployed to progressively evaluate more expressive and complex text-classification techniques. Specifically, the strategy was to begin with fast,

interpretable baselines to establish performance expectations, and then to escalate to more sophisticated neural architectures. As stated, across all model selections, three text feature variations were assessed: 1) title only, 2) description only, and 3) title+description to quantify the predictive contribution of each textual dimension.

4.1 Multinomial Naïve Bayes

Multinomial Naïve Bayes served as the initial baseline classifier due to its computational efficiency and well-documented historical classification performance on sparse text representations. The model assumes conditional independence among terms given the class label – an assumption that is rarely true in practice but is surprisingly effective when implemented in a TF-IDF matrix representation.

Thus, to operationalize this model, TF-IDF matrices were constructed using up to 30,000 features and unigram+bigram tokenization, with English stop words removed to reduce noise. Moreover, the smoothing parameter, alpha, was explored as a tunable metric at values of 0.1, 0.5, and 1.0, enabling the model to settle on an efficient balance between over- and under-penalizing rare terms (Figure 3).

```
Confusion Matrix (Accuracy 0.9126)

         Prediction
  Actual    World   Sports Business Sci/Tech
   World     2691      108      116       85
  Sports       21     2957       12       10
Business       94       18     2607      281
Sci/Tech      110       16      178     2696
```

Figure 3

In terms of results, combining title and descriptive text features, paired with an alpha term set to 0.1 (only slight penalty applied, which keeps the values close to their raw terms originally in the matrix), yielded the best classification performance on the holdout test set within this subset of models.

As expected, the Naïve Bayes model provided a fast, reliable benchmark against which more complex models could later be compared.

4.2 Regularized Multinomial Logistic Regression

Multinomial Logistic Regression was selected as a discriminative linear classifier that assigns weights as values to the TF-IDF matrix as an indicator of evidence that the term(s) within a document belong to a given class. Thus, each word contributes independently to the classification task, enabling the model to effectively handle many sparse features and automatically learn the significance of words within each class. In effect, Logistic Regression

often returns an improvement in accuracy over Naïve Bayes in TF-IDF spaces. Key tunable parameters included regularization strength, C, and penalty type (L1, L2, or elastic net).

A two-stage grid search was conducted, in which the objective of the first stage was to identify an optimal regularization strength under the lbfgs solver and L2 penalty term type for an array of the top 20,000 most frequent unigram text variants across all articles (Figure 4).

Logistic Regression

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=1)
```

```
pipe = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words='english', max_features=20000, ngram_range=(1,1)))
     ('log', LogisticRegression(max_iter=2000, random_state=1))
])
```

```
log_grid ={
        # L2 (ridge) with lbgfs
  "log__solver": ['lbfgs'], 'log__penalty': ['l2'], 'log__C': [0.001, 0.1, 1, 10]}
```

```
grid = GridSearchCV(pipe, log_grid, cv=cv, scoring='accuracy', n_jobs=-1, verbose=2)
```

Figure 4

Then, using the same constraints and optimal regularization configuration identified in stage 1, the TF-IDF grid space was expanded to identify the optimal number of features/tokens to include in the vocabulary, as well as the optimal n-gram range: unigrams only or unigrams and bigram vocabulary feature representations. (Figure 5).

```
tfidf_grid = [{
    'log__C': [best_log_params['log__C']],
    'log__penalty': [best_log_params['log__penalty']],
    'log__solver': [best_log_params['log__solver']],
    'tfidf__max_features': [20000, 30000, 40000],
    'tfidf__ngram_range': [(1,1), (1,2)]
}]
```

Figure 5

The optimal parameter configuration settings for the best logistic regression variation was: C = 1, ridge regularization and the lbfgs solver, top 40000 most frequently found tokens across all articles to be included in the vocabulary, with that vocabulary consisting of both unigrams and bigrams.

Again, the title-and-text description variation of the logistic regression model achieved the highest accuracy on the holdout test set, with an overall accuracy metric almost 5% higher than the optimal title-only variation (Figure 6).

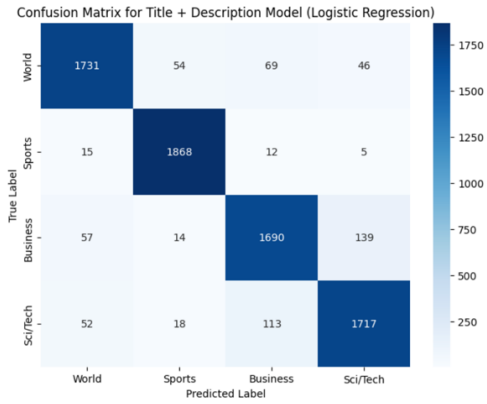

Figure 6

The classification matrix for the optimal logistic regression model demonstrates balanced performance across all four classes, with relatively low misclassification rates and deviation in accuracy between them.

4.3 Recurrent Neural Network (LSTM variant)

A Recurrent Neural Network (RNN), with an LSTM layer that compresses the entire sequence of text embeddings into a single vector of length 128 before being fed into the classification head layer, was selected as an initial deep-learning classifier to capture the temporal and contextual structure of the articles' vocabulary. Unlike Naïve Bayes and Logistic Regression, which treat documents as unordered collections of terms, the LSTM layer processes text as an ordered sequence of tokens, up to 200 sequential time steps of tokens in my specific use case, enabling it to learn context and semantic patterns across each article. This contextualized hidden state then serves as the basis for classification. Although more computationally intensive than linear approaches, due to the natural structure, number of nodes and layers, tunable parameters, and backpropagation, the RNN model provides a baseline test assessment of whether incorporating word order and sequential context yields measurable improvements over traditional discriminative and generative TF-IDF models for classifying news articles.

In assessing the models performance, validation accuracy plateaued rather quickly after only running through three epochs (Figure 7).

Figure 7

As we can see in assessing the learning curves, the model was able to generalize rather quickly and validation performance doesn't appear to improve with additional training in this specific use case. Performance was certainly competitive with logistic regression but not meaningfully superior, if at all, especially when taking into consideration the longer and more computationally intensive training times. Thus, the RNN offered limited value compared to traditional linear models for classifying the AG news articles.

4.4 Transformer (RoBERTa)

A pretrained RoBERTA Transformer model was selected as the state-of-the-art deep learning classifier for the news article classification experiment. The RoBERTa encoder architecture is similar to BERT, as they are both masked language models that specialize in predicting masked tokens given context; however, the RoBERTa architecture is more modern, and trained longer and with more data, which is why its deep layer structure and weights were chosen to serve as the foundation backbone layer for the classification task. This learner is capable of leveraging contextualized language representations far beyond what generative and discriminative TF-IDF linear-based models or even sequential RNN models offer, given that, for every token position, a vector representation that encodes surrounding context is returned and passed into the classification head. Whereas LSTMs process inputs token-by-token and linear models treat the tokens as independent features, RoBERTa uses multi-head self-attention to evaluate semantics and relationships between all tokens in each article simultaneously, enabling the model to learn nuanced grammatical relationships, long-range dependencies of word, and contextual cues before classifying each article.

During preprocessing, each article is first tokenized into subword units and then passed through the pretrained RoBERTA Transformer backbone, which produces rich contextual embeddings that serve as the foundation for classification. A lightweight feed-forward classification head is then fine-tuned on top of the backbone layer to adapt the model to the AG News domain. Moreover, the backbone layer was set to frozen to ensure its precise pretrained weights weren't inadvertently updated during training, and the backpropagation of gradients focused on enhancing the learning through the dense feedforward network layered

in on top of it. Early stopping was also set to monitor validation loss after each run through the dataset, with training instructed to terminate and roll back to the epoch with 'best validation performance' if validation loss failed to improve over two consecutive epochs (Figure 8).

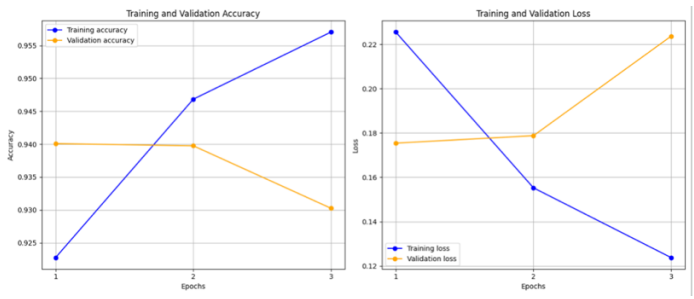

Figure 8

Although the backbone layer is technically trainable, both a full fine-tuning of both the backbone layer and feedforward network and a cross-validation grid search for hyperparameter tuning were decided against due to computational cost (approximately 45 minutes of training per epoch) and likely only marginal gains in terms of classification accuracy for this specific use case. Moreover, given the facts only three epochs were run through before the early stopping criteria terminated training due to failed improvement in validation loss, the overall experiment represented a transfer learning oriented and lightly fine-tuned learner rather than a deeply optimized Transformer architecture.

Despite limited tuning, the RoBERTa Transformer returned the strongest and most robust results – a test accuracy of 94%, balanced across all four classes of articles – thus confirming that pretrained encoder-based Transformers provide meaningful performance gains over traditional linear and RNN models, albeit at substantially higher computational cost to train (Figure 9).

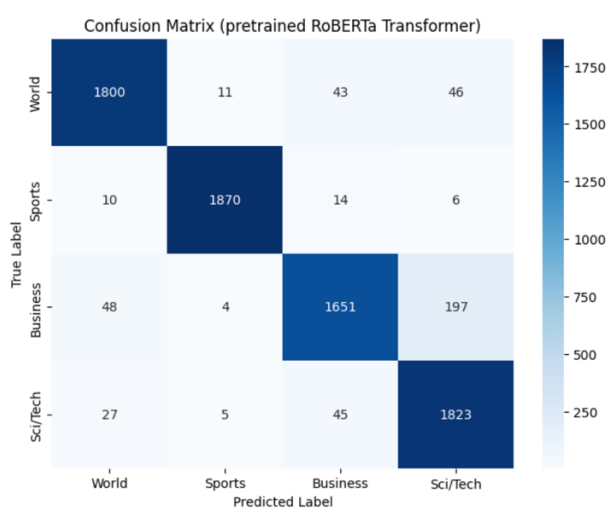

Figure 9

5. Discussion

5.1 Text Field Choice

Across all model variants, a consistent pattern emerged regarding the correlation between predictor selection and predictive performance. Models trained solely on titles performed reasonably well, but with noticeable lag relative to models trained on descriptions. This reflects the inherent limitations of headlines with fewer tokens and thus less signal. Headlines, after all, are intentionally concise, and sometimes crafted to grasp views rather than clarity in meaning. In contrast, descriptive summary fields provide much deeper narratives, context, topical signal, and more diverse, descriptive vocabulary, enabling classification models to form stronger decision boundaries during training. However, the best results were achieved when both title and descriptive fields were combined, confirming that the two fields offer complementary signals: the title effectively introduces the central theme of the article, while the description elaborates on the details of the article and help distinguish meaning. This pattern held across both Naïve Bayes and Logistic Regression, rather affirmatively, which is partially why training the RNN and Transformer on only the text or description fields was sidestepped as an observational strategy.

5.2 Why Frequent Global N-grams Were not Removed

TF-IDF analysis surfaced several globally frequent unigrams, bigrams, and trigrams, such as “new,” “new york,” “afp afp,” “ap ap,” and “href http www,” that appear to be ambiguous, technical article artifacts rather than meaningful linguistic features. Although it may seem intuitive to remove these terms, especially before training more advanced models as they don’t default to removing stop words, doing so would offer limited benefit and introduce unnecessary complexity (Figures 10 & 11).

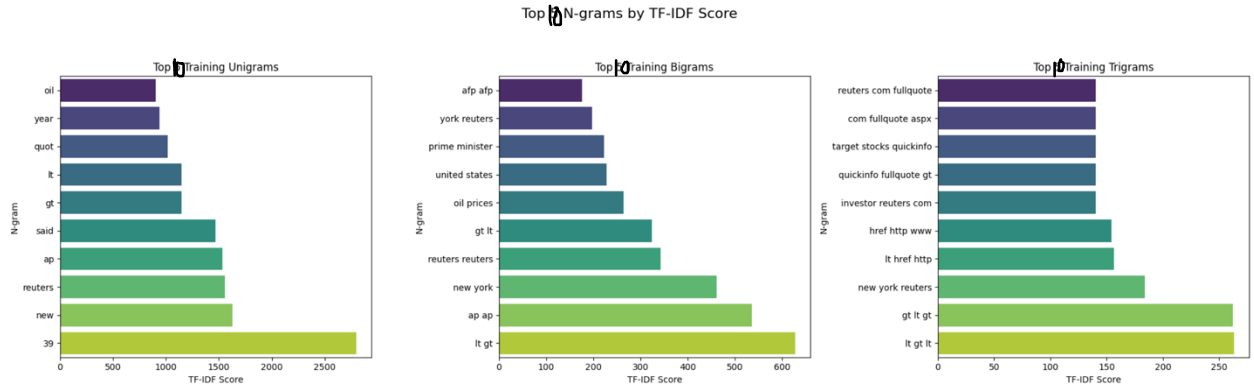

Figure 10

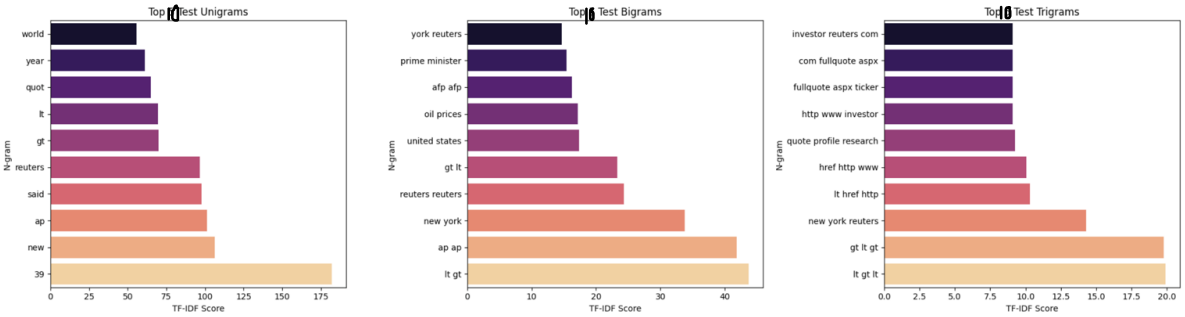

Figure 11

In the case of linear models such as Naïve Bayes and Logistic Regression, the TF-IDF weighting scheme naturally down-weights globally common terms through the inverse document frequency (IDF) component, meaning their influence on the model is negated. Thus, removing these tokens manually would produce marginal predictive improvements, if any. For RNNs and Transformers, the case for removal is different. These architectures learn internal representations of tokens through contextualized embeddings that allow them to largely ignore uninformative tokens without the explicit need to pre-filter them out. In the case of RoBERTa, altering the token stream would break the assumptions of the pretrained vocabulary and subword segmentation. Especially when taking into consideration the strong and robust performance achieved by both models, especially the Transformer, and the additional overhead required to implement custom token filtering, the decision not to remove these frequent n-grams aligns with the general modeling decision to avoid unnecessary engineering complexities when the expected benefit is marginal or ambiguous.

5.3 Training a Traditional Artificial MLP Neural Network

While MLPs can, in theory, capture nonlinear relationships that linear models cannot, they are not well-suited to handle extremely high-dimensional sparse inputs. Thus, training an MLP on tens of thousands of TF-IDF features, although attempted, quickly became computationally inefficient, especially when combined with cross-validation, and was quickly cast aside as an effective modeling choice. During experimentation, training times escalated rapidly, with a single fold going well over fifteen minutes to train. Because I was already confident that the performance on the test set would be rather mediocre and overfit, and because I simply wanted to assess how the architecture performed relative to the simpler linear models, I decided to end the experiment prematurely. Logistic regression, in comparison, had already returned promising accuracy at a fraction of the training cost. The marginal, if any, incremental accuracy did not justify the additional runtime, infrastructure demands, and model complexity for this text classification task.

6. Business-Oriented Recommendations

6.1 Recommended Baseline Model for Operational Deployment

For organizations in demand of a practical and cost-effective classification-based solution, the TF-IDF Logistic Regression model presents the most balanced approach. The modeling architecture provides high accuracy, trains in minutes or even seconds depending on hardware, and is fully explainable if need be, a critical requirement for organizations working in environments that demand full transparency in their automated decision-making rules.

6.2 Considering Advanced Architectures

The RNN model, while functional and accurate, does not offer a compelling argument to choose it over a simpler Logistic Regression architecture for this task. Its longer training time and similar performance make it a less attractive option for practical deployment.

However, the RoBERTa Transformer appears particularly well-suited for high-stakes business applications, in which classification accuracy is a determining factor for outcomes that are decided at the margin. Its superior performance makes it an excellent candidate for premium content curation or systems that must reliably distinguish an array of meticulous and layered topics. It's worth noting, though, that deploying such a model requires careful pre-planning in determining computational demand, available infrastructure, and latency constraints. Organizations should only consider adopting Transformers into their business model when the incremental accuracy gains justify investment.

7. Conclusion

Ultimately, this project highlights that model selection, while certainly a technical decision, is also a strategic, forward-looking decision. Organizations must carefully assess accuracy and interpretability demands, infrastructure requirements, and cost when undertaking model deployment endeavors. For many domains, it is likely that a finely-tuned logistic regression model offers the best balance of performance and efficiency. For high-stakes environments operating at the margin, a Transformer-based solution is certainly reasonable. The structured, comparative modeling sequence enforced in this project emphasizes the importance of aligning analytical rigor with real-world business needs; thus, providing a clear roadmap for future text classification initiatives.